

Academic Session 2024-2025
Autumn Semester

Computer Organization Lab

Verilog Assignment

22nd October 2024

Group -29

More Aayush Babasaheb
Devanshu Agrawal

(22CS30063)
(22CS30066)

Instruction Encoding- R-Type

Operation	Opcode	Function Code
ADD	000000	000000000
SUB	000000	000000001
NOR	000000	000000010
AND	000000	000000100
OR	000000	000000101
XOR	000000	000000110
NOT	000000	000000111
SLT	000000	000001000
SGT	000000	000001001
SLL	000000	000001010
SLR	000000	000001011
SAR	000000	000001100
INC	000000	000001101
DEC	000000	000001110
HAM	000000	000001111
MOVE*	000000	000010000
CMOV	000000	000011000

*While encoding instruction, only Rs and Rd will be provided. Rt will be taken to be 0000

I-Type & J-Type

Operation	Encoding
ADDI	010000
SUBI	010001
NORI	010010
LUI	010011

ANDI	010100
ORI	010101
XORI	010110
NOTI	010111
SLRI	011011
SARI	011100
SLTI	011000
SGTI	011001
LUI	010011
HAMI	011111
BR	100000
BMI	100001
BPL	100010
BZ	100011
LD	100100
ST	101000
HALT	111110
NOP	111111

Encoding Schemes-

R-type instructions

Opcode	Source Register 1 (Rs)	Source Register 2 (Rt)	Destination Register (Rd)	Shift Amount	Function Code
6 bits	4 bits	4 bits	4 bits	5 bits	9 bits

I-type instructions

Opcode	Source Register (Rs)	Destination Register (Rt)	Immediate Value
6 bits	4 bits	4 bits	18 bits

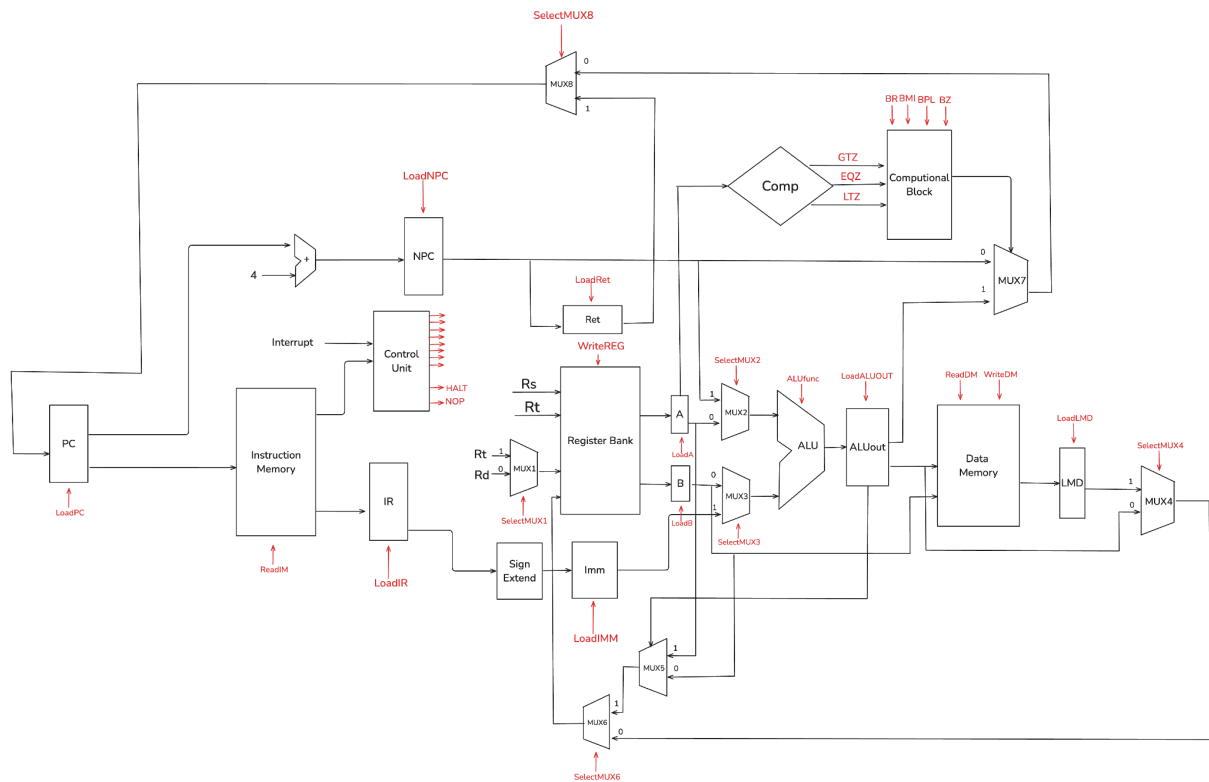
J-type instructions

Opcode	Immediate Value
6 bits	26 bits

Relevant Assumptions–

1. Instruction memory and data memory are treated as separate memories.
2. The stack register is made part of the Register Bank.

DataPath Diagram–



Computational Block Specification

Outputs whether to perform a branch or not.

$$\text{Computational Block Output} = \text{BR} + \text{BMI} * \text{LTZ} + \text{BPL} * \text{GTZ} + \text{BZ} * \text{EQZ}$$

Return Register

The Return Register is a part of the register bank and thus can be loaded and used as a normal register. Still, the special connection allows us to Load the NPC value at the time of a function call into that register which can then be stored in a registered addressed memory to be used in recursive calls and calls to other functions.

In the diagram, we have shown the RET block separately for ease of understanding.

Control signals

1. loadPC
2. loadIR
3. loadNPC
4. loadA
5. loadB
6. writeREG
7. loadIMM
8. SelMUX1 (for Rr and Rd)
9. SelMUX2 (for A or PC)
10. SelMUX3 (for B or IMM)
11. SelMUX4 (for store or saving aluout data in regbank)
12. SelMUX6 (for cmove or store output data)
13. SelMUX8 (for loading pc with new address or return address)
14. ALUfunc
15. WMFC
16. loadALUout
17. loadRet
18. loadLMD
19. readIM
20. readDM
21. writeDM
22. 4 branching signals to control branch type (BR, BMI, BPL, BZ)
23. HALT
24. NOP
25. DONE

Micro-routine specifications

The compilation of all micro-routine specifications has been done in the spreadsheet listed below indicated with checkboxes specifying if the signal is set to active or not.

Link- [The spreadsheet](#)

Control Path Diagram

